

16/6/26

DIGITAL DESIGN WITH FPGA

MC (Microcontroller)	FPGA (Field-Programmable gate array)	ASIC (Application specific integrated ckt.)
It is sequential exe	It has parallel process	same as FPGA
Low cost	high speed	highest speed
General purpose (software level)	General purpose (hardware level)	specialised purpose

RTL design [Two hardware description language]

- verilog HDL → system verilog (advanced)
- VHDL

& → logical AND

| → logical OR

^ → logical xor

~ → logical NOT.

Design Entry

Verilog HDL, VHDL, system verilog

↓
next go to simulation

↓
synthesis

converts RTL to gate level netlist (how gates are interconnected)

↓
Physical design (implementation) → mapped to target FPGA

verilog code for AND GATE:

module → keyword.

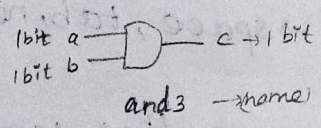
module and3 (a, b, c);

input a, b;

output c;

assign c = a & b;

endmodule



if a = 1 bit
b = 2 bit } not combined declare as 1 bit

keyword
in data flow
modelling

module logic_gates (a,b, c_and, c_or, c_xor, c_not);

input a,b;

output c_and, c_or, c_xor, c_not;

assign c_and = a & b;

assign c_or = a | b;

assign c_xor = a ^ b;

assign c_not = ~a;

endmodule.

18/6/26

UNIT-1 : VERILOG HDL

LEXICAL CONVERSIONS: (fundamental rules)

→ Case sensitive.

1. Identifiers → Name of the object

→ first character must be alphabet & underscore

variable

module

→ should not give space.

2. keyword. (reserved word) cannot used as identifier

Eg: input, output, inout, module, end module, etc.

3. white spaces. (all spaces are removed) while compilation

→ space, tab, new line (\n), command line //

multiple line command

4. Numbers

Eg: a = 4'b0010 (4 bit binary)

a = 4'd2

14 bit decimal

h → hexadecimal, o → octal.

module and port definitions.

↓
design or Blank box.

To define a module:

module module_name (i/p & o/p variable);

input a, b;

output sum, carry;

} description

endmodule

(or)

module module_name (input a, b, output sum, carry);

— — — } description.

endmodule

ports [interface to the outside world]

type i/p

→ o/p

→ inout → bidirectional port (both i/p & o/p)

operators: OF VERILOG HDL:

1. Arithmetic operator: +, -, *, /, **, %

a = 4'b0100, b = 4'b0010

Eg:

assign c = a + b; → 4'b0110

assign d = a - b; → 4'b0010

assign e = a * b; → 4'b1000

assign f = a / b; → 4'b0010

remainder

2. Logical operator: !(logical NOT), logical AND (&&),

logical OR (||)

Eg:

a = 4'b0100

b = 4'b0010

c = 4'b0110

d = 4'b0111

eg: ~~sign~~ if (a > b && c > d)

y = 1'b1

else

y = 1'b0

(88)

o/p: y = 1'b0

(11)

o/p : y = 1'b1

eg: if (!xst)

x = d;

else

x = 1'b0;

If condition false

o/p is d

else

o/p is 0

3. Equality operator.

== → equality operator

!= → inequality operator

=== → case equality check bit by bit (2 & 2 a 40)

!== → case inequality.

(identical)

1
2 = 2

alphabet → assign person

Let $a_1 = 4'b1010$

; $b_1 = 4'b1010$ equal op (1)

inequality op (0), case equality (1), case inequality (0)

$a_2 = 4'b1010$

; $b_2 = 4'b1010$ eq op (1)

ine op (0), case eq (0), case neq (1)

$a_3 = 4'b1001$

; $b_2 = 4'b1111$

equ op (0), ine (1), case eq (0), case neq (1)

4. relational operators: (eg)

>, <, >=, <=, ==.

5. Bitwise operator:

Bitwise AND &

Bitwise OR |

" NOT ~

" EXOR ^

" XOR ~^

$a = 4'b0010$

$b = 4'b1111$

$c = a \& b$

o/p = $4'b0010 = c$

6. Conditional operator: (?:) Ternary operator

Eg: $c = a > b ? \text{if true } 1'b1 : \text{if false } 1'b0$

$a = 4'b0010, b = 4'b1111$

o/p: $c = 1'b0$

7. Shift operator:

$\ll \rightarrow$ left shift

$\gg \rightarrow$ right

$\lll \rightarrow$ Arithmetic LS

$\ggg \rightarrow$ Arithmetic RS

Eg: $a = 4'b1011$

$c = a \ll 2$; 1100 (shift factor 2)

$d = a \gg 3$; 0001

$c = a \lll 2$; 1100

$d = a \ggg 2$; 1110 (sign bit $\rightarrow$ MSB)

8. Reduction operator: (operand will be one) & o/p is 1 bit

&, |, ^, ~&, ~|, ~^

Eg: $c = \&a$

if $a = 4'b0111$

9. Concatenation operator: (combining) { }

assign $z = \{2'b10, 2'b01, 2'b11\}$

assign $z = \{a, b, c\}$ if $a = 2'b10$

$b = 2'b11$

$c = 2'b01$

o/p

$z = 6'b101101$

10th replication operator: { f }₄

assign z = { 4 { a } }

If a = 2'b10

↳ a will replicate 4 times

o/p

z = 8'b10101010

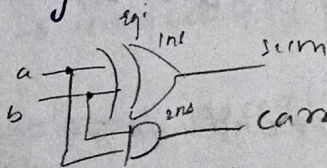
19/6/26

Levels of modeling in Verilog HDL

- Gate level modeling (describe the ckt in terms of logic gates)
- Data flow modeling
- Behavioural modeling
- structural modeling.

1. Gate level [low level description]

describing the ckt in terms of logic gates



critical path is a to carry

```
module ha(a, b, sum, carry);
```

```
input a, b;
```

```
output sum, carry;
```

```
xor (sum, a, b);
```

```
and (carry, a, b);
```

```
endmodule
```

Advantage

Basic

→ suitable for simple cts

→ Basic primitive optimize.

critical path →

ilp takes long time to become o/p.

disadv

→ not suitable for larger

2. Data flow

→ Describing the circuit using boolean logic.

→ assign (keyword) used

module half_adder (a, b, sum, carry);

input a, b;

module ha (input a, b, output sum, carry);

assign sum = a ^ b;

assign carry = a & b;

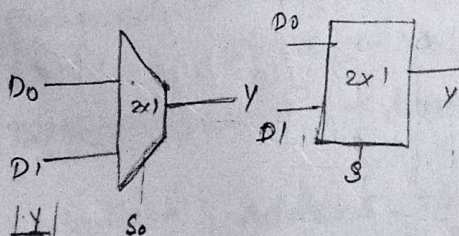
endmodule;

advantages

→ suitable for bigger cks compare to gate level

→ more readability

2x1 multiplexer using
gate level



D ₀	D ₁	Y
0	0	0
0	1	0
1	0	1
1	1	1

$$Y = D_0 \bar{S} + D_1 S$$

module mul (input D₀, D₁, S,

output Y, Y₁, Y₂);

not (X, S)

and (Y₁, D₀, X)

and (Y₂, D₁, S)

OR (Y, Y₁, Y₂);

endmodule

2x1 multiplexer using

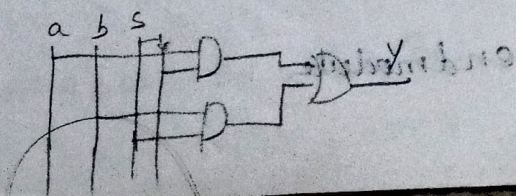
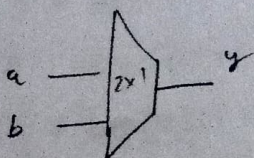
Data flow

module mul (input D₀, D₁, S,
output Y)

assign Y = (D₀ & S) | (D₁ & S);

endmodule

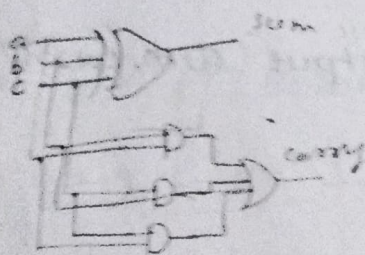
2x1 multiplexer $Y = a\bar{S} + bS$



full adder using gate level

$$\text{sum} = a \oplus b \oplus c$$

$$\text{carry} = \bar{a}b + \bar{b}c + \bar{c}a$$



```

module fa (input a,b,c
  output sum,carry);
  wire x,y,z;
  xor (sum,a,b,c);
  and (carry,x,a,b);
  and (y,b,c);
  and (z,c,a);
  or (carry,x,y,z);
endmodule

```

full adder using data flow



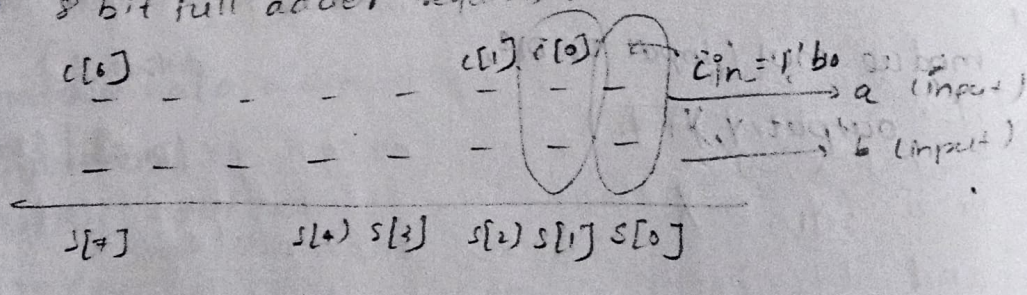
```

module fa (input a,b,c,
  output sum,carry);
  assign sum = a^b^c;
  assign carry = (a&b) |
    (b&c) | (c&a);
endmodule.

```

23/6/26

8 bit full adder requires for 8 bit addition



```

module fa (input a,b,c,output sum,carry);
  assign sum = a^b^c;
  assign carry = ((a^b)&c) | (a&b);
endmodule

```



```
module adders(input [7:0]a,b, input cin,
    output [7:0]s, output carry);
```

```
wire [6:0]c;
```

```
fa z1 (a[0], b[0], cin, s[0], c[0]);
```

```
fa z2 (a[1], b[1], c[0], s[1], c[1]);
```

```
fa z3 (a[2], b[2], c[1], s[2], c[2]);
```

```
fa z4 (a[3], b[3], c[2], s[3], c[3]);
```

```
fa z5 (a[4], b[4], c[3], s[4], c[4]);
```

```
fa z6 (a[5], b[5], c[4], s[5], c[5]);
```

```
fa z7 (a[6], b[6], c[5], s[6], c[6]);
```

```
fa z8 (a[7], b[7], c[6], s[7], carry);
```

```
endmodule
```

testbench code → functional verification for the logic

```
module adders_tb;
```

```
reg [7:0]a;
```

```
reg [7:0]b;
```

```
reg cin;
```

```
wire [7:0]s;
```

```
wire carry;
```

```
adders dut (a, b, cin, s, carry);
```

```
initial
```

```
begin
```

```
a = 8'b10010010, b = 8'b00100011, cin = 1'b0;
```

```
#10 after 10 ns. from 0.
```

```
a = 8'b10101111, b = 8'b00111101, cin = 1'b1;
```

```
#5
```

initial
block
start
t=0.

$a = 8'b00001111, b = 8'b11110000, \text{cin} = 1'b0;$

end

endmodule.

2 bit multiplier

module mul2 (input a, b, output

sum, carry);

assign sum = a ^ b;

assign carry = a & b;

endmodule.

module mul (input [1:0] a, b, output [3:0], product)

assign p[0] = a[0] & b[0];

assign z[0] = a[0] & b[0];

assign z[1] = a[0] & b[1];

assign z[2] = a[1] & b[0];

assign z[3] =

ha q1 (z[0], z[1], p[1], z[3]);

ha q2 (z[3], z[2], p[2], p[3]);

endmodule.

Testbench

module mul_tb;

reg [1:0] a;

reg [2:0] b;


```

wire [3:0] p;
module mul(a, b, p);
    a = 1'b0, b = 1'b1;
#10
    a = 1'b1, b = 1'b1;
#15
    a = 1'b0, b = 1'b0;
#5
    a = 1'b1, b = 1'b0;
end
endmodule.

```

24/6/26

System task: (\$) [only in testbench prog]

→ Simulation commands

- similar to (printf)
- \$display → to display strings, exp, num, immediately in active region
 - \$monitor → to display strings, exp, num, upon to display signal value upon it changes
 - \$
 - \$write → to display str, exp, num without adding new line
 - \$strobe → to display str, exp or num at the end of current time slot.
 - \$finish → used to complete the simulation.

```

module display_tb;

```

```

    reg [3:0] d1, d2;

```

```

    initial begin

```

```

        at 10 d1 = 4; d2 = 3;

```

```

        #5 d1 = 2; d2 = 3;

```

```

    end

```


Initial begin

#15 ^{no monitor}
time of two \$display ("At time %t: { \$display → d₁=%d,
d₂=%d }");

\$monitor ("At time %t: { \$monitor → d₁=%d,
d₂=%d }");

\$write ("At time %t: { \$write → d₁=%d,
d₂=%d }");

(-: %t, %d → format specifier)

O/p:

At time t=0: \$display d₁=4, d₂=3.

At time t=0: \$monitor d₁=4, d₂=3

At time t=0: \$write d₁=4, d₂=3, At time t=5:

\$monitor d₁=2, d₂=3.

%t, %d → Format specifier (%)
time ↓ return decimal
%h → hexa, %o → octal, %b → binary.

Compiler directives:

→ Instructions given to compiler to how to handle the program.

Start with (') not end with (;)

1. 'define → To define text macros

Eg: 'define count = 100

2. 'include adder.v added a file
3. 'if def To check if particular macro is defined or not

eg: if def clk
 { display → 1
 else
 { display → 0

endf.

4. 'timescale

$$\frac{1\text{ns}}{1\text{ns}} \rightarrow \frac{\text{Time unit}}{\text{Time precision (steps)}}$$

eg: a=1, b=2;

#0.48 a=0, b=3;

#0.51 a=4, b=6;

o/p:

t=0 a=1, b=2

t=0.48 x 1ns = 0.48205; a=0, b=3
 t=0 (rounded)

t=1, a=4, b=6

0.51 x 1ns = 0.51 → round to 1

if timescale is

$$\frac{10\text{ps}}{1\text{ps}}$$

t=0 x 10ps = 0ps = a=1, b=2

t=0.48 x 10ps = 4.8ps = 5ps = a=0, b=3

t=0.51 x 10ps = 5.1ps → 10ps = a=4, b=6

30/6/26

Behavioural modelling [high level description]

Procedural statements

→ initial

→ always

initial block

- used for initializing variables
- used in write the test bench code
- executed only once [an]
- start at $t=0$.
- multiple no. of initial blocks allowed
- nested initial block not allowed.

initial
begin
—————
————— } for multiple
————— } lines
end

Eg: initial

executed
one by
one
at $t=5$

begin

$a = \#5 \text{ 'b1' ; } t=5, a=1$
 $b = \#10 \text{ 'b0' ; } t=15, b=0$
 $c = \#15 \text{ 'b1' ; } t=30, c=1$

end

initial

begin

$a = \#5 \text{ 'b0' ; } t=5, a=0$
 $b = \#10 \text{ 'b1' ; } t=15, b=1$

end.

two initial block
is independent
of time

multiple
blocks

always block

- will get repeated throughout the simulation
- Have multiple no. of always blocks allowed
- nested always block not allowed.

```
module dff (input clk, rst, d, q);
```

```
input clk, rst;
```

```
input d;
```

```
output reg q;
```

```
always @ (posedge clk)
```

```
begin
```

```
if (rst)
```

```
q = 1'b0;
```

```
else
```

```
q = d;
```

```
end
```

```
endmodule
```

Blocking assignment: ($=$) current line execution blocks the following line execution

Non Blocking assignment ($=>$)

execution of current line will not affect the execution of following line

Eg: initial
begin

```
a = #5 1'b0;
```

```
b = #10 1'b1;
```

```
c = #15 1'b0;
```

end

dp: t=5, a=0
t=15, b=1
t=30, c=0

for blocking assignment

Eg. for non-blocking

initial

begin

a <= #5 1'b1;

b <= #10 1'b0;

c <= #15 1'b1;

end

o/p:

t=5, a=1

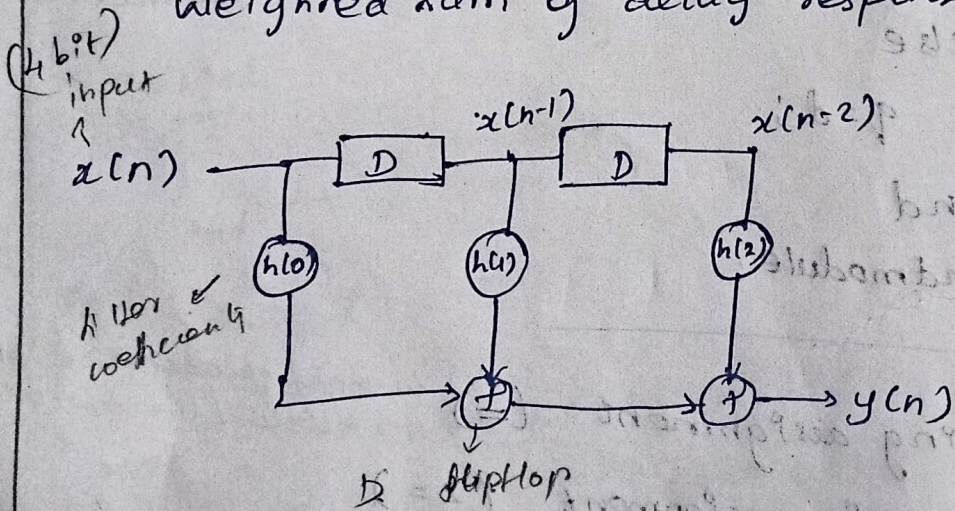
t=10, b=0

t=15, c=1

} all three b, c, d at
t=0 (independently)

FIR Filter:

weighted sum of delay responses.



module dff(clk, rst, d, q);

input clk, rst;

input [3:0] d;

output [3:0] q;

always @(posedge clk)

begin

if (rst)

q <= 4'b0;


```

else
    qz = d;
end
endmodule.

```

Top module:

```

module fir (input [3:0] x, h0, h1, h2, output [7:0]
            y, rstclk);

```

```

    reg [7:0] y;

```

```

    wire [3:0] x_1, x_2;

```

```

    wire [7:0] y_1, y_2, y_3;

```

```

    dff d1 (clk, rst, x, x_1);

```

```

    dff d2 (clk, rst, x_1, x_2);

```

```

    assign y_1 = x * h0;

```

```

    assign y_2 = x_1 * h1;

```

```

    assign y_3 = x_2 * h2;

```

```

    always @ (posedge clk)

```

```

    begin

```

```

        if (rst)

```

```

            y = 8'b0;

```

```

        else

```

```

            y = y_1 + y_2 + y_3;

```

```

    end

```

```

endmodule.

```

2/7/26

Selection statements

ternary operator

→ assign c = (exp) ? statement 1 : statement 2;

→ if, else

eg:

```

if begin

```

```

end

```

```

begin

```

```

end

```

```

if (rst)

```

```

    q = 1'b01

```

```

else

```

```

    q = 1'b01

```

↑
simple selection statement

used only
in procedural
blocks
(initial & always)

used in procedural block
→ case statement:

case (expression)
not begin
and end. _____

endcase

eg. for 4:1 mux.
case (s)

2'b00: y = a;

2'b01: y = b;

2'b10: y = c;

2'b11: y = d;

endcase.

single
line

if you have
one begin
and

2'b00: y = a;
fix in
end.

2'b01: y = b;

begin
and.

Looping Statement (eg) ~~⊗~~
used in procedural block

→ 4 statements

(for, while, forever, repeat)

not
support
for
verilog HDL

non-synthesizable constraints

not support for FPGAs in
verilog hdl.

use for
benchmark,
simulation

For

for (i = 0, i < 10, i = i + 1)

begin _____ c[i] = a[i] + b[i];

end.

while initial i = 0;

while (i < 10);

begin

c[i] = a[i] + b[i];

i = i + 1;

end.

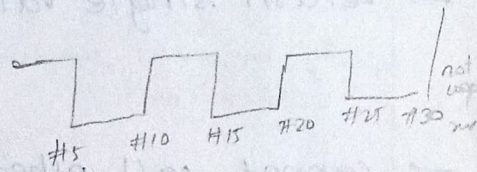
Forever

loop runs indefinitely, it won't
terminate automatically.

when, we the condition $\$ finish$ then the simulation will stop thus the this forever loop also be come to end.

Eg:
`initial
begin
forever clk = ~clk;
#5;
end
initial
begin
#30;
$ finish; // stop the simulation`

both initial block run independently



stop the forever loop at 30 ns

repeat
`initial
begin
repeat (N)
end
initial
begin
clk = ~clk;
#5
end.`

repeat CLK pulse
repeat signal

Functions and Tasks

→ Function or task is a group of statement used to perform a specific task. Both of them can be called at various points of the program.

→ used to break large code into smaller pieces to make it easier to

read and debug.

Function

Task

- | Function | Task |
|---|----------------------------------|
| → cannot contain simulation delay (Eg: #100, #15) | can contain simulation delay. |
| → return single value
Eg: half adder, sum carry, only return 'sum' | returns multiple values |
| → cannot call other task | can call other task or function. |

Eg: ^{keyword}

function function_name;

input

~~end~~

endfunction.

Syntax
for
function

Eg: 4:1 mux using function

function mux4;

input a,

input b,

input c,

input d,

input [1:0] s;

case (s)

2'b00 : mux4 = a;

2'b01 : mux4 = b;

2'b10 : mux4 = c;

2'b11 : mux4 = d;

endcase
endfunction.

random line in Topmodule.

main module
y = mux4 (a, b, c, d, S); // fun call to sub module

Task

Syntax

Task task_name;

ilp

o/p

endtask

Ex: half adder

Task ha;

input a,

input b,

output sum,

output carry;

sum = a ^ b;

carry = a & b;

end task.

ha(a, b, sum, carry); // task called

BCD TO SEVEN SEGMENT DECODER.

0 → 9 → input [4:0]

output a, b, c, d, e, f, g

module DEC [

input [4:0] a

output [7:0] b

);

	a	b	c	d	e	f	g	
0	1	1	1	0	0	1	3	
1	0	1	0	0	1	1	4	
2	1	0	1	1	0	1	5	
3	1	0	1	1	0	0	6	
4	1	1	1	0	0	0	7	
5	1	1	1	1	1	1	8	
6	1	1	1	1	0	1	9	

~~a = 8010
#10 ns.~~

```
module bcdto7seg (input [3:0]s, input clk,  
                  output [6:0]y );
```

```
always @ (posedge clk) without use clk  
begin. always @ (posedge clk);
```

```
    case (s)
```

```
        4'b0000 : y = 7'b1111110;
```

```
        4'b0001 : y = 7'b0110000;
```

```
        4'b0010 : y = 7'b1101101;
```

```
        4'b0011 : y = 7'b1111001;
```

```
        4'b0100 : y = 7'b0110011;
```

```
        4'b0101 : y = 7'b1011011;
```

```
        4'b0110 : y = 7'b1011111;
```

```
        4'b0111 : y = 7'b1110000;
```

```
        4'b1000 : y = 7'b1111111;
```

```
        4'b1001 : y = 7'b1111011;
```

```
    endcase
```

```
end
```

```
endmodule.
```


module counter (input clk, rst, enable, mode,
output [3:0] count);

always @ (posedge clk)

begin

if (rst)

count = 4'b0;

else

begin

if (enable)

begin

if (mode == 1'b0)

count = count + 1;

else

count = count - 1;

end

end

end

end module

reset if else is allowed.

Program for digital clk

input clk, rst,

sec, min, hour;

module digclk (input clk, rst, output sec,
min, hour,);

always @(posedge clk)

begin

if (rst)

sec = 6'b0;

min = 6'b0;

hour = 4'b0;

else

end sec = sec + 1;

begin

if (sec == 60)

min = 1'b1;

sec = 0'b0;

else

sec = sec + 1;

end

begin if (min == 60)

hour = 0'b1;

min = 0'b0;

else

sec = sec + 1;

end

endmodule

main module

module digclk (input clk, rst,
output reg [5:0] sec, min,
output reg [3:0] hour);

always @(posedge clk)

begin

if (rst)

sec = 6'b0;

min = 6'b0;

hour = 4'b0;

else

begin

if (sec == 6'b111111)

begin

sec = 6'b0;

min = min + 1;

if (min == 6'b111111)

begin

min = 6'b0;

hr = hr + 1;

end

end

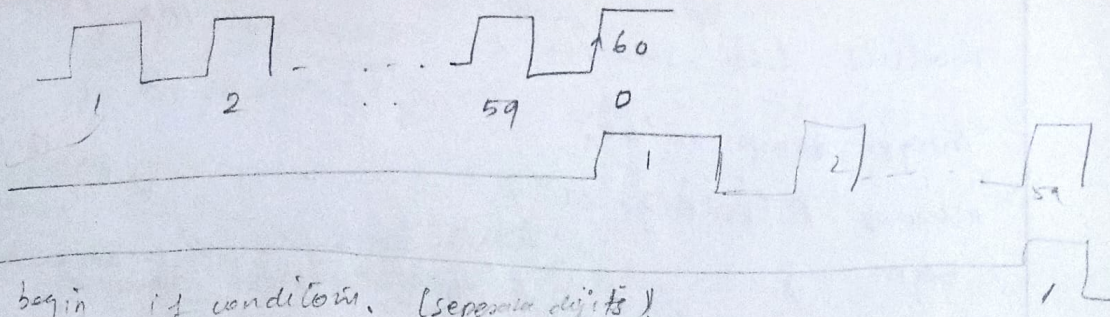
else

sec = sec + 1;

end

endmodule

sec
min
hr



begin if condition. (separate digits)

begin digit separation mem.

if (sec <= 6'b001001) [< 9]

sec_o = 6'b000 ;

sec_t = 6'b001001 ;

end

begin

if (sec > 6'b001001) && (sec < 6'11)

sec_o = sec - 10 ;

sec_t = 6'b000001 ;

end

begin

if (sec > 11) && (sec < 21)

sec_o = sec - 20 ;

sec_t = 6'b000010 ;

end

begin

if (sec > 21) && (sec < 31)

sec_o = sec - 30 ;

sec_t = 6'b000011 ;

end

begin

if (sec < 31) && (sec > 41)

sec_o = sec - 40

sec_t = 6'b000100 ;

end

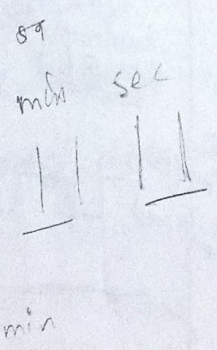
begin

if (sec < 41) && (sec > 59)

sec_o = sec - 50

sec_t = 6'b000101 ;

end



50 MHz → crystal
freq → 5 MHz

1 Hz = 15

clock div
module

module CLK, slowclk) discussed at 11:12

integer temp = 24'60;

always @ (posedge clk)

begin

temp = temp + 1;

slowclk = temp[23];

endmodule

to 2 bit

1 μs clk

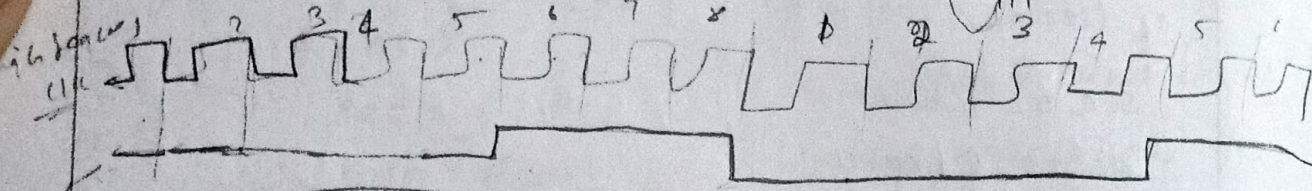
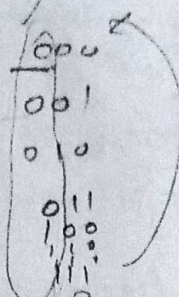
Initial

incr

+1

+1

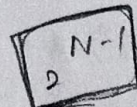
pulse m3 at slowclk



slow clk

8 μs clk

More in 3 bit



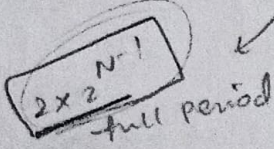
$$2^{N-1} = 2^{3-1}$$

half period

$$2^2 = 4$$

{after 4 clk pulse}

the state is toggle



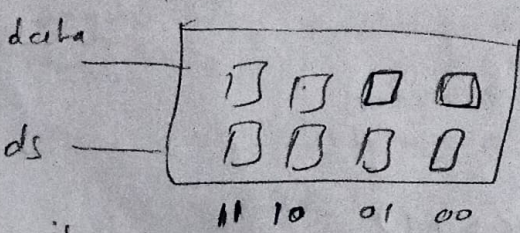
in the case of 4 bit

half period

$$2^{N-1} = 2^{4-1} = 2^3$$

$$\text{full period} = 2 \times 2^3$$

24 μs clk



digit selection ilp

will not run in slow clk should be in high frequency for display all frequent

module ds (clk, sec-0, sec-t, min-0, min-t, o/p y)

ds = 0

always @ (posedge clk)

begin ds = ds + 1

case (ds)

00 : y = sec-0;

01 : y = sec-t;

(i) 0+1 = 1 → 01 → sec-t

(ii) 1+1 = 2 → 10 → min-0

(iii) 2+1 = 3 → 11 → min-t

(iv) 3+1 → 100 → come 0
↓
becz it is 3 bit

10 : y: min-0;

11 : y: min-t;

endcase

Program for digital clock.

RTC-Top.

```
module rtc_top (clk, rst, sec, min, hr, sec_o, sec_t,  
min_o, min_t, hr_o, hr_t, sec_o_seg, sec_t_seg,  
min_o_seg, min_t_seg, hr_o_seg, hr_t_seg, data, digit);
```

Input clk, rst;

output [5:0] min, sec;

output [3:0] hr, sec_o, sec_t, min_o, min_t, hr_o, hr_t,
_(12 format) digit;

output [7:0] sec_o_seg, sec_t_seg, min_o_seg, min_t_seg,
hr_o_seg, hr_t_seg, data;

wire clk1, clk2;

clkdiv c1 (clk, clk1);

clkdiv c2 (clk, clk2);

```
rtc r1 (clk1, rst, sec, min, hr, sec_o, sec_t, min_o, min_t,  
hr_o, hr_t, sec_o_seg, sec_t_seg, min_o_seg, min_t_seg,  
hr_o_seg, hr_t_seg);
```

```
digit_selection d1 (sec_o_seg, sec_t_seg, min_o_seg,  
min_t_seg, hr_o_seg, hr_t_seg, clk2, rst, digit, data);
```

endmodule

NRIC

```
module atc (clk, rst, sec, min, hr, sec_o, sec_t,  
min_o, min_t, hr_o, hr_t, sec_o_seg, sec_t_seg,  
min_o_seg, min_t_seg, hr_o_seg, hr_t_seg);
```

```
input clk, rst;
```

```
output reg [5:0] sec, min;
```

```
output reg [3:0] hr;
```

```
output reg [3:0] sec_o, sec_t, min_o, min_t, hr_o, hr_t;
```

```
output reg [7:0] sec_o_seg, sec_t_seg, min_o_seg,  
min_t_seg;
```

```
output reg [7:0] hr_o_seg, hr_t_seg;
```

```
always @ (posedge clk)
```

```
begin
```

```
if (rst)
```

```
begin
```

```
sec = 6'b0;
```

```
min = 6'b0;
```

```
hr = 4'b0;
```

```
end
```

```
else
```

```
begin
```

```
if (sec == 6'b111011)
```

```
begin
```

```
sec = 6'b0
```

```
min = min + 1;
```

```
if (min == 6'b111011)
```

```
begin
```

```
min = 6'b0;
```

```
hr = hr + 1;
```

```
end
```

```
end
```

```
else
```


sec = sec + 1;

end.

'begin

(9)

if (sec <= 6'b001001)

begin sec_o = sec[3:0]; sec_t = 4'b0000; end

else if (sec > 6'b001001 && sec <= 6'b010011)

begin sec_t = 4'b0001; sec_o = sec - 6'b001010; end

else if (sec > 6'b010011 && sec <= 6'b011101)

begin sec_t = 4'b0010; sec_o = sec - 6'b010100; end

else if (sec > 6'b011101 && sec <= 6'b100111)

begin sec_t = 4'b0011; sec_o = sec - 6'b011110; end

else if (sec > 6'b100111 && sec <= 6'b110001)

begin sec_t = 4'b0100; sec_o = sec - 6'b101000; end

else if (sec > 6'b110001 && sec <= 59)

begin sec_t = 4'b0101; sec_o = sec - 6'b110010; end

if (min <= 6'b001001)

begin min_o = min_t[3:0]; min_t = 4'b0000; end

else if (min > 6'b001001 && min <= 6'b010011)

begin min_t = 4'b0001; min_o = min - 6'b001010; end

else if (min > 6'b010011 && min <= 6'b011101)

begin min_t = 4'b0010; min_o = min - 6'b010100; end

else if (min > 6'b011101 && min <= 6'b100111)

begin min_t = 4'b0011; min_o = min - 6'b011110; end

else if (min > 6'b100111 && min <= 6'b110001)

begin min_t = 4'b0100; min_o = min - 6'b101000; end

else if (min > 6'b110001 && min <= 59)

begin min_t = 4'b0101; min_o = min - 6'b110010; end

and

begin

case (sec-o)

d p g f e / d c b a

1 0 1 1 1 1
1 0 0 0 1 2

1 - OFF
0 - ON

4'b0000 : sec-o-seg = 8'b11000000;

4'b0001 : sec-o-seg = 8'b11111001;

4'b0010 : sec-o-seg = 8'b10100100;

4'b0011 : sec-o-seg = 8'b10110000;

4'b0100 : sec-o-seg = 8'b10011001;

4'b0101 : sec-o-seg = 8'b10010110;

4'b0110 : sec-o-seg = 8'b10000010;

4'b0111 : sec-o-seg = 8'b10000000;

4'b1000 : sec-o-seg = 8'b10000000;

4'b1001 : sec-o-seg = 8'b10010000;

endcase

case (sec-t)

4'b0000 : sec-t-seg = 8'b11000000;

4'b0001 : sec-t-seg = 8'b11111001;

4'b0010 : sec-t-seg = 8'b10100100;

4'b0011 : sec-t-seg = 8'b10110000;

4'b0100 : sec-t-seg = 8'b10011001;

4'b0101 : sec-t-seg = 8'b10010110;

4'b0110 : sec-t-seg = 8'b10000010;

4'b0111 : sec-t-seg = 8'b11111000;

4'b1000 : sec-t-seg = 8'b10000000;

4'b1001 : sec-t-seg = 8'b10010000;

endcase

case (min-o)

4'b0000 : min-o-seg = 8'b11000000;

4'b0001 : min-o-seg = 8'b11111001;

4'b0010 : min-o-seg = 8'b10100100;

4'b 0011 : min_o_seg = 8'b 10110000;
 4'b 0100 : min_o_seg = 8'b 10011001;
 4'b 0101 : min_o_seg = 8'b 10010110;
 4'b 0110 : min_o_seg = 8'b 10000010;
 4'b 0111 : min_o_seg = 8'b 11111000;
 4'b 1000 : min_o_seg = 8'b 10000000;
 4'b 1001 : min_o_seg = 8'b 10010000;

endcase

case (min_t)

4'b 0000 : min_t_seg = 8'b 11000000;
 4'b 0001 : min_t_seg = 8'b 11111001;
 4'b 0010 : min_t_seg = 8'b 10100100;
 4'b 0011 : min_t_seg = 8'b 10110000;
 4'b 0100 : min_t_seg = 8'b 10011001;
 4'b 0101 : min_t_seg = 8'b 10010110;
 4'b 0110 : min_t_seg = 8'b 10000010;
 4'b 0111 : min_t_seg = 8'b 11111000;
 4'b 1000 : min_t_seg = 8'b 10000000;
 4'b 1001 : min_t_seg = 8'b 10010000;

endcase

case (hr_o)

4'b 0000 : hr_o_seg = 8'b 11000000;
 4'b 0001 : hr_o_seg = 8'b 11111001;
 4'b 0010 : hr_o_seg = 8'b 10100100;
 4'b 0011 : hr_o_seg = 8'b 10110000;
 4'b 0100 : hr_o_seg = 8'b 10011001;
 4'b 0101 : hr_o_seg = 8'b 10010110;
 4'b 0110 : hr_o_seg = 8'b 10000010;
 4'b 0111 : hr_o_seg = 8'b 11111000;
 4'b 1000 : hr_o_seg = 8'b 10000000;
 4'b 1001 : hr_o_seg = 8'b 10010000;

endcase


```
case (hr_t)
```

```
4'b0000 : hr_t_seg = 8'b11000000;
```

```
4'b0001 : hr_t_seg = 8'b11110001;
```

```
endcase
```

```
end
```

```
end
```

```
endmodule
```

```
// Digit selection
```

```
module digit_selection (sec_o_seg, sec_t_seg,
```

```
min_o_seg, min_t_seg, hr_o_seg, hr_t_seg,
```

```
clk, rst, digit, data);
```

```
input [7:0] sec_o_seg, sec_t_seg, min_o_seg,
```

```
min_t_seg, hr_o_seg, hr_t_seg;
```

```
input clk, rst;
```

```
output reg [3:0] digit;
```

```
output reg [7:0] data;
```

```
reg [1:0] sel;
```

```
always @ (posedge clk)
```

```
begin
```

```
if (rst)
```

```
sel = 2'b00;
```

```
else
```

```
begin
```

```
case (sel)
```

```
2'b00 : begin digit = 4'b110; data = sec_o_seg; end
```

```
2'b01 : begin digit = 4'b1101; data = sec_t_seg; end
```

```
2'b10 : begin digit = 4'b1011; data = min_o_seg; end
```

```
2'b11 : begin digit = 4'b0111; data = min_t_seg; end
```


endcase

sel = sel + 2'b01;

end

end

endmodule

// CLK DIV 1

module clkdiv1 (clk, clk2);

input clk;

output reg clk2;

reg [26:0] c;

initial c = 2'b0;

always @ (posedge clk)

begin

c = c + 1;

clk2 = c;

end

endmodule

// CLK DIV

module clkdiv (clk, clk1);

input clk;

output reg clk1;

reg [10:0] c;

initial c = 11'b0;

always @ (posedge clk)

begin

c = c + 1;

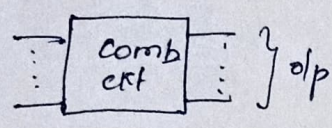
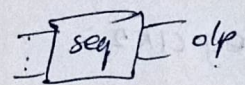
clk1 = c;

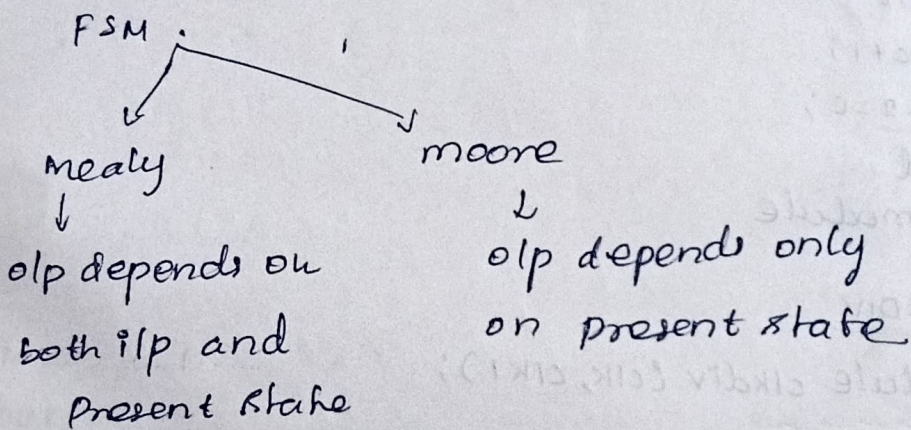
end

endmodule.

23/7/26

FSM $\rightarrow$ abstract model for describing seq. circuit.

comb	seq
o/p $\rightarrow$ present i/p's  Eg: full adder	o/p $\rightarrow$ present i/p & Present state  Eg: counter



steps for designing FSM:

1. Define a problem
2. Draw a state transition diagram (pictorial representation)
3. Draw a state table (tabular representation)
4. Remove redundant states
5. state assignments
6. Find flipflop for realization of logic ckt
7. Draw the excitation table

7. Find boolean function. (for flip-flop 1/ps)

8. Draw the logic diagram.

Eg. Sequence Detector (1010)

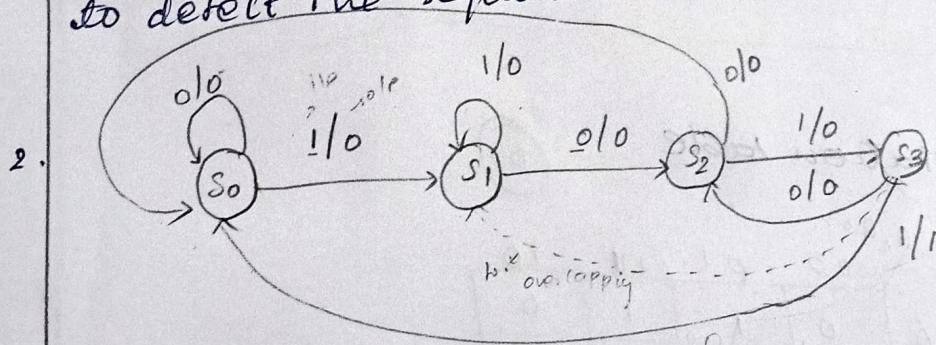
non overlapping :

10100001010
 00010000001

overlapping (1011)

10110110001011
 00010010000001

1. Consider a non-overlapping sequence detector to detect the sequence 1011.



3. State table

X	Present state	Next state	o/p
0	S ₀ 00	00 S ₀	0
1	S ₀ 00	01 (S ₁)	0
0	S ₁ 01	10	0
1	S ₁ 01	01	0
0	S ₂ 10	00	0
1	S ₂ 10	11	0
0	S ₃ 11	10	0
1	S ₃ 11	00	1

4. No redundant states
5. State assignments are done
6. flipflop logic circuit

X	ps $a_1 a_2$	D1	D0	o/p
0	0 0	0	0	0
1	0 0	0	1	0
0	0 1	1	0	0
1	0 1	0	1	0
0	1 0	0	0	0
1	1 0	0	1	0
0	1 1	1	0	0
1	1 1	0	0	1

Excitation table

(D1)

X	$a_1 a_0$	00	01	10	11
0		0	1	1	0
1		0			1

$$D_1 = X \bar{a}_1 \bar{a}_0 + a_0 X$$

X	$a_1 a_0$	00	01	11	10
0		1	1		0
1		1	1		1

(D0)

$$D_0 = X \bar{a}_1 + X a_1 \bar{a}_0$$

(cont)

$$D_0 = X \bar{a}_0 + X a_1$$